WPT Day 16 Morning

Abhishek Purohit · Thu 05 Nov 26 · Schema-based form validation with React Hook Form, a Yup `productValidationSchema`, `@hookform/resolvers` as the middleman, MUI `TextField`s wrapped in `Controller`, and a standalone Tailwind login screen (API + token left for the evening)

Table of Contents

1. React Hook Form
2. Yup Schema
3. Resolvers
4. Controller
5. Login Screen
6. Combined Worked Example
7. Quick Revision Checklist
8. Practice Exercises
9. Practice Answers

React Hook Form

Problem: four fields already collect data — they still accept anything

Day 15 Morning built the create form in `WPT Day 15 Evening Codes/crud-ui/src/pages/CreateProduct/CreateProduct.tsx` (that evening folder is the **starting** project this lecture continues). Four MUI `TextField`s, one `useState<IFormData>`, one `handleChange`, one `handleSubmit` that `preventDefault`s and `POST`s to `/product`. That is enough to *send* a product. It is not enough to *refuse* a bad one.

The lecturer listed the rules this form actually needs:

Field	Rules the class named
Name	Required. At least **3** characters.
Description	Required. At least a few characters; later locked as **max 100**.
Price	Required. Must be **numeric**. Must be **greater than 0**.
Quantity	Required. Must be **numeric**. Must be **greater than 0**. Must be an **integer** — `2.5` and `3.6` are illegal.

The quantity rule got the lecture's own picture: *साढ़े तीन की board कहाँ से लाओगे, आधा तोड़ के थोड़ी ना दोगे* — you are not going to break a board in half to sell three-and-a-half of them. Quantity is a

count of whole items.

Without those rules the API will happily insert `"ab"`, a negative price, or `qty: 2.5`. The toast will still say **Product Created**. The catalogue is then lying.

Why writing the rules as if-else is the wrong job

The class already did this the hard way in core HTML: read each field, write a condition, write another condition, write another. The lecturer walked the same ladder for **one** field:

1. Is there a value at all? (`required`)
2. Is it a number? (`numeric`)
3. Is it greater than 0?
4. Is it an integer, or a fractional value like `2.5` ?

Four questions, four `if` s — and that is **quantity alone**. Name, description and price each get their own ladder. The lecturer: *हमलोग कोड लिख रहे थे... but that is really a very time-taking process. काफी ज़्यादा time लगेगा.*

There is a second cost on top of the `if` s. The Day 15 form also owns the **plumbing**:

Job you currently do by hand	**Where it lives in the evening `CreateProduct.tsx`**
Hold the four values	`useState<IFormData>({ name: "", description: "", price: "", qty: "" })`
Copy each keystroke into state	`handleChange` → `setFormData({ ...formData, [event.target.name]: event.target.value })`
Stop the browser reload	`event.preventDefault()` inside `handleSubmit`
Clear the boxes after a good POST	`setFormData({ name: "", description: "", price: "", qty: "" })`
Decide which message to show	You would write this next, as more `if` s

React Hook Form exists so that **none** of those five rows are your code any more.

Mechanism: one hook owns the form

React Hook Form is a third-party package. The lecturer called it another module that lets you *handle our form really very easy as compared to the from-the-scratch approach*. Install it on the **frontend** (`crud-ui`), not on `crud-apis` :

```
npm i react-hook-form yup @hookform/resolvers
```

The same three names are in `WPT Day 16 Morning Codes/notes.txt`. After install they appear in `WPT Day 16 Morning Codes/crud-ui/package.json` as `react-hook-form`, `yup`, and `@hookform/resolvers`.

The library's one core hook is `useForm`. Documentation line the lecturer read out: *the core hook for managing a form — returns all the methods and state for the registration, validation and submission.*

```
import { Controller, useForm } from "react-hook-form";
```

You call it once in `CreateProduct`. You do **not** call `useState` for the four fields. You do **not** write `handleChange`. The Day 16 file leaves both of those as comments so you can see what was deleted:

```
// const [formData, setFormData] = useState<IFormData>({ name: "", description: "",
price: "", qty: "" });

// const handleChange = (event: ChangeEvent<HTMLInputElement | HTMLTextAreaElement,
Element>) => {
//     setFormData({ ...formData, [event.target.name]: event.target.value });
// }
```

What you **do** take out of the hook:

Piece from `useForm`	Job
`control`	*Connect each field to the form* — the lecturer's own comment in `CreateProduct.tsx`
`reset`	Empty every registered field. Replaces `setFormData({ name: "", … })`
`formState: { errors }`	The messages from the schema, keyed by field name
`handleSubmit`	Library function you hang on the `<form>`'s `onSubmit`

Class typed the hook with the same interface Day 15 already had, from `WPT Day 16 Morning Codes/crud-ui/src/pages/CreateProduct/types.ts`:

```
export interface IFormData {
    name: string;
    description: string;
```

```
    price: string;
    qty: string;
}
```

```
const {
    control,
    reset,
    formState: { errors },
    handleSubmit
} = useForm<IFormData>({
    resolver: yupResolver(productValidationSchema)
});
```

`resolver` is not React Hook Form inventing the rules. It is how the hook is told **which schema to read**. That wiring is the next two sections. `useForm` can also take `defaultValues` (the lecturer pointed at the update-page initial values). Create does not need them — blank is correct.

Your submit handler is no longer an event handler

Day 15's function was named `handleSubmit` and took a browser `SubmitEvent`. That name now belongs to the **library**. Class renamed the own function to `onSubmit`.

The `<form>` (still an MUI `Box component="form"`) does this:

```
<Box
    component="form"
    className="bg-white p-6 shadow-lg rounded-xl"
    onSubmit={handleSubmit(onSubmit)}
    noValidate
    sx={{ marginTop: "30px" }}
>
```

Read that `onSubmit` prop as a **pipeline**, not as "call my function":

```
  user clicks Add Product
           |
           ▼
  <form onSubmit={handleSubmit(onSubmit)}>
           |
           ▼
  React Hook Form handleSubmit
     1. run the schema
     2. if invalid → fill `errors`, do not call you
     3. if valid   → call your onSubmit(formData)
           |
           ▼
  onSubmit(formData)   ← already the four values, already typed IFormData
     apiClient.post("/product", formData)
```

```
    showSuccessToast("Product Created !")
    reset()
```

Two things you used to write by hand are gone:

- **No `event` argument.** The lecturer: *this function is not going to take an event. Now this function is directly going to have the form data with it.*
- **No `preventDefault`.** React Hook Form already stopped the browser reload.

```tsx
const onSubmit = async (formData: IFormData) => {
    console.log(formData);
    try {
        const response = await apiClient.post("/product", formData);
        console.log(response);
        console.log(response.data);
        showSuccessToast("Product Created !");
        reset();
    } catch (error) {
        console.log(error);
        showErrorToast("Product Creation Failed !");
    }
};
```

`apiClient` is still `WPT Day 16 Morning Codes/crud-ui/src/api-client/ApiClient.ts` (`baseURL` `http://127.0.0.1:7500`). Toasts are still `WPT Day 16 Morning Codes/crud-ui/src/utils/ToastUtil.ts`. Those files did not change this morning. Only the **way the form reaches them** changed.

`noValidate` is an instruction to the **browser**: do not run HTML5 built-in bubbles (`required` on the DOM, native number checks). The lecturer: *I don't want that form अपने खुद का कोई browser का default validation call करे... all the job is now going to be performed with the help of React Hook Form and Yup.*

> **Exam:** React Hook Form owns form *data*, *change*, *submit* and *error display*. You do not create field state. `handleSubmit` is the library's; your function is `onSubmit(formData)` with no `event` and no `preventDefault`. After a good POST you call `reset()`, not `setFormData({ …empty })`.

> **Pitfall:** Leaving your function named `handleSubmit` and writing `onSubmit={handleSubmit}` (no call, no wrapper) skips the schema. The library never gets a chance to validate. The form attribute must be `onSubmit={handleSubmit(onSubmit)}`.

What React Hook Form does *not* do by itself

It will manage values and show errors. It will **not** invent the rules. The lecturer was explicit: one way to validate is still "write multiple if-else." The way this class is taking is **schema-based validation** — you write *rules*, the hook *reads* the schema. Creating that schema is Yup. Connecting Yup to the hook is `@hookform/resolvers`. Those are the next two sections, not extra features of `useForm`.

Yup Schema

Problem: React Hook Form still needs something to read

A hook that "validates against a schema" is useless until a schema exists. The lecturer: *in a schema, you just have to define certain rules… schema में आपको conditions नहीं लिखनी होती है. You just have to specify some rules only.*

Background: a **schema** here is a description of a valid object — "name is a required string of at least 3 characters" — not a list of `if` statements that you run yourself. A validator library *interprets* that description and returns either "ok" or a per-field message.

Yup is one such library. The lecture audio writes it as "Yep"; the npm package, the import, `notes.txt` and `package.json` are all `yup`. Zod was named as another option. Class chose Yup.

Mechanism: `object({ … })` whose keys are the input `name`s

Yup's docs example the lecturer opened is an object with keys `name`, `age`, `email`, `website`, `createdOn`. Left-hand side = the input field. Right-hand side = the chain of rules for that field:

Docs field	Rules he read off the page
`name`	string, required
`age`	number, required, positive, integer
`email`	email rule
`website`	url rule
`createdOn`	date-style rule

The product form is the same shape with **our** four keys. Those keys are not labels and they are not ids. They are the `name` of each field — `name`, `description`, `price`, `qty`. The lecturer: *ये जो left-hand side पे keys हैं वो आपका same होना चाहिए जो input field का name है. नहीं तो validation काम नहीं करेगा.*

He refused to dump the schema into `CreateProduct.tsx`. *इसी TSX फाइल के अंदर मैं नहीं define कर रहा हूँ because it will again mix up few of the things.* New file, same folder:

`WPT Day 16 Morning Codes/crud-ui/src/pages/CreateProduct/ValidationSchema.ts`

```
import { number, object, string } from "yup";

export const productValidationSchema = object({
    name: string()
        .required("Name is required")
        .min(3, "Minimum 3 characters are required"),

    description: string()
        .required("Description is required")
        .max(100, "Maximum 100 characters allowed"),

    price: number()
        .typeError("Only numeric value is allowed")
        .required("Price is required")
        .positive("Price cannot be -ve or 0"),

    qty: number()
        .required("Quantity is required")
        .integer("Quantity must be integer")
        .min(1, "Quantity must be atleast 1")
});
```

`object()` is the function that *defines the shape*. Each rule function also takes the **message the user should see** when that rule fails. The lecturer: *form तो submit नहीं होगा, validation चलेगा, but we have to show certain error messages as well. वो error message कहाँ पे configure करोगे? That is in these functions only.*

Rule-by-rule, in the order class wrote them

Name — first field, `name="name"`.

- `string()` — this value is text.
- `.required("Name is required")` — empty is a failure. Message on a blank Add Product click.
- `.min(3, "Minimum 3 characters are required")` — one or two characters is a different failure. Class typed a single character in the demo and watched this message replace the required one; at three characters the red state disappeared.

Description — key must be `description`, not `desc`.

- First pass in class used `required` plus a `min`. Final file keeps `required` and a `max(100)`. One character is allowed; the 101st is not.
- Message: `"Maximum 100 characters allowed"`. Demo: keep typing until the counter crosses 100, red helper text appears.

Price — a **number**, not a string.

- `.required("Price is required")`
- `.positive("Price cannot be negative")` on the first write. `positive` means **greater than zero**. Zero is *not* positive.
- Live demo then typed a **string** into Price. Yup's default type message is technical: *price must be a `number` type*. Class went back and added `.typeError("Only numeric value is allowed")` so a letter shows a human sentence.
- Live demo then typed `0`. `positive` rejected it, but the message still said "cannot be negative." They reworded it to `"Price cannot be -ve or 0"` — the string in the saved file.

Quantity — number, required, **integer**, at least one.

- `.integer("Quantity must be integer")` — `2.5` fails. This is the board-in-half rule, as a function.
- First they discussed `.positive()`. Final file uses `.min(1, "Quantity must be atleast 1")` so zero is also refused, with a message that says what to do (give at least one), not only what went wrong.
- In the demo the lecturer said to put the same `typeError` on quantity as on price. The **saved** `ValidationSchema.ts` does not have that line on `qty` — only on `price`. If a student types `abc` in Quantity they may still see Yup's default type sentence unless they add it.

Format the chains on new lines. The lecturer: *अगर later on we have to debug it, तो एक single line में read करना is really a very tough job.*

When each message appears

A message is shown **only when its own rule is broken**. Empty name → required message, not the min-3 message. Three-character name that is otherwise fine → no `name` key in `errors` at all.

```
Add Product clicked, name box empty
    name rule "required" fails
    errors.name.message = "Name is required"

User types "K"
    required now passes
    min(3) fails
    errors.name.message = "Minimum 3 characters are required"

User types "Kur"
```

```
both name rules pass
errors.name is undefined
red border + helper text go away
```

Exam: Schema keys = input `name` attributes (`name` , `description` , `price` , `qty`). Yup is rules, not `if` s. `string()` / `number()` then `.required()` , `.min()` , `.max()` , `.positive()` , `.integer()` , `.typeError()` . Each function's string argument is the helper text for **that** failure.

Pitfall: A schema key `quantity` will never validate a field named `qty` . React Hook Form looks up errors by the Controller's `name` . Mismatch = silent skip, or a field that never turns red.

Resolvers

Problem: two libraries that do not know each other

React Hook Form does not create Yup schemas. Yup does not register MUI fields or call `onSubmit` . They are **separate packages**. The lecturer: *एक और module चाहिए जो कि इन दोनों को link करने का काम करेगा.*

That module is `@hookform/resolvers` . Role of each of the three installs:

Package	What it is responsible for	What it is *not* responsible for
`react-hook-form`	Form data, change, submit, putting messages on screen	Writing the rule list
`yup`	The rule list (`productValidationSchema`)	Talking to the `<form>` or to MUI
`@hookform/resolvers`	The **middleman** — turns a Yup schema into something `useForm` can read	Either of the jobs above, on its own

Zod (or another schema library) would still need a resolver. The resolver package ships one helper per schema library. Class imported **`yupResolver`** .

Mechanism: `resolver: yupResolver(schema)`

```
import { yupResolver } from "@hookform/resolvers/yup";
import { productValidationSchema } from "./ValidationSchema";
```

Pass it as the `resolver` option of `useForm`. That is the entire link:

```
useForm<IFormData>({
    resolver: yupResolver(productValidationSchema)
});
```

The lecturer: *React Hook Form is now going to read this product validation schema.* After a submit (and, in this setup, as the user edits), the hook runs the schema, fills `formState.errors`, and only then — if nothing failed — calls `onSubmit`.

`formState` can also tell you `isValid`, `isSubmitting`, `disabled`. Class kept it simple and destructured **`errors` only**, *because मुझे errors को further display करना होगा.*

The shape he drew on the board (also left as a comment in `CreateProduct.tsx`):

```
errors = {
    description: { message: "…" },
    name:        { message: "…" }
}
```

A key exists **only when that field is currently failing**. A valid field is missing, not `{ message: "" }`. That is why the UI checks `errors.name` before reading `.message`.

> **Exam:** Three packages, three jobs. `yupResolver(productValidationSchema)` is the one line that connects Yup to React Hook Form. `errors` is an object of *failing* fields; each value has a `message` taken from the schema function that failed.

> **Pitfall:** Importing `yupResolver` from `@hookform/resolvers` (no `/yup`) is the wrong path. Class imported from `@hookform/resolvers/yup`. Installing only two of the three packages also fails: no Yup → no schema; no resolver → `useForm` cannot read the schema; no React Hook Form → nothing to hang the resolver on.

Controller

Problem: React Hook Form has never heard of MUI `TextField`

This is the step the lecturer called *the toughest job... और ये केवल one-time setup है.*

React Hook Form knows how to talk to a plain `<input>`. It does **not** know what your UI library called the box. The lecturer's list:

What you might render	Who owns that component
`<input>`	The browser
MUI `TextField`	`@mui/material` — this project
Some other library's `TextInput`	Daisy UI, or whoever

The people who wrote React Hook Form cannot know which of those you picked, or which prop on that component is `name`, or which event is `onChange`. So they do not ask you to put `name=` on the MUI field. They ask you to wrap **whatever** you render inside **their** component: `Controller`.

The lecturer: *Controller component is a placeholder that is going to contain your input field... Controller is just a wrapper of your MUI input field.*

Mechanism: `name` + `control` + `render`

`Controller` comes from the same `react-hook-form` import as `useForm`. Every field that must be validated gets one.

```
<Controller
    name="name"
    control={control}
    render={(controller) => {
        const { field } = controller;
        return (
            <TextField
                {...field}
                label="Name"
                variant="outlined"
                fullWidth
                error={!!errors.name}
                helperText={errors.name?.message}
            />
        );
    }}
/>
```

Three props, three jobs:

Prop	What you pass	Why
`name`	`"name"` / `"description"` / `"price"` / `"qty"`	This is now the field's identity. It must match the Yup key. You no longer put `name=` on `TextField`.
`control`	The `control` from `useForm`	*Connect each field to the form.* One form, one `control` object, four Controllers sharing it.
`render`	A function that **returns** the real widget	React Hook Form still does not know about `TextField`. You decide, here, what the user sees.

`render` receives an object. The lecturer named three bags inside it:

Bag	What is in it	Did class use it?
`field`	Props the widget needs: `name`, `value`, `onChange`, `onBlur`, `ref`	**Yes** — this is the one they spread
`fieldState`	That field's own error / dirty / touched	Not this morning
`formState`	The whole form's state	They already took `errors` from `useForm`

Two ways to write the function, both in the same file:

Name and description — take the whole controller, then destructure `field`, then `return (` … `)`:

```
render={(controller) => {
    const { field } = controller;
    return (
        <TextField {...field} label="Name" /* ... */ />
    );
}}
```

Price and quantity — destructure in the argument and use the arrow-function implicit return (no `{ }`, no `return`). The lecturer: *बार-बार ऐसा return भी ना लिखना पड़े तो वही arrow function का benefit use कर लो. curly braces हटाओ... return की जरूरत नहीं.*

```
render={({ field }) => (
    <TextField {...field} label="Price" /* ... */ />
)}
```

Same behaviour. The second is shorter once you are comfortable.

Why `{...field}` and not `name=` / `onChange=` / `value=`

`field` already **is** those props. Writing them out by hand (`name={field.name}` `onChange={field.onChange}` ...) works until you miss one. The lecturer: *उनको कब तक यहाँ पे mention करोगे? उससे अच्छा... spread कर देता हूँ field को ताकि जो बच्चे भी props हैं वो यहाँ पर मिल जाएं.*

So on `TextField` you keep only what MUI still needs from **you**:

Still yours	Why
`label`	"Name", "Description", "Price", "Quantity"
`variant="outlined"`	Same look as Day 15
`fullWidth`	Fill the Grid cell
`error={…}`	Red border — MUI's boolean
`helperText={…}`	The sentence under the box

You **drop** `name`, `value` and `onChange`. Controller owns them. The Day 15 `value={formData.name}` / `onChange={handleChange}` lines are the ones the comments at the top of the file replaced.

MUI's error state is a boolean; `errors.name` is an object

MUI's own docs (the lecturer opened them): `error` is `true` or `false`. When `true`, the outline goes red. The sentence is a **separate** prop, `helperText`.

`errors.name` is **not** a boolean. When the name rule is failing it is `{ message: "Name is required" }`. When the name rule is passing it is `undefined`.

Class wrote the readable form first, then the shortcut they actually shipped:

Style	Code	Who it is for
Beginner ternary	`error={errors.name ? true : false}`	Still in a comment on the Name field
Double-not	`error={!!errors.name}`	What they left on all four fields

The analogy he gave, slowly:

1. `errors.description` when valid is `undefined`.
2. MUI will not accept `undefined` as `error` — it wants a boolean.

3. One `!` turns `undefined` into `true` (negation). That is the **wrong** boolean.
4. A second `!` cancels the negation and leaves you with **`false`**, still a boolean. `!!{ message: "…" }` is `true`.

Helper text is the `.message` inside that object, with optional chaining so a valid field does not explode:

```
helperText={errors.name?.message}
```

If `errors.name` is missing, `?.` stops, `helperText` is `undefined`, MUI shows nothing under the box.

```
errors (only failing keys are present)

  {
    name:  { message: "Minimum 3 characters are required" },
    price: { message: "Only numeric value is allowed" }
  }

  Name TextField
      error={!!errors.name}                 → true  → red outline
      helperText={errors.name?.message}     → "Minimum 3 characters are required"

  Description TextField
      error={!!errors.description}          → false → normal outline
      helperText={errors.description?.message} → undefined → no sentence
```

Header and theme on the same page

Validation is the new work. The page also picked up last night's Tailwind look, copied from `WPT Day 16 Morning Codes/crud-ui/src/pages/ProductsList/ProductsList.tsx` (the list header was already there from Day 15 Evening).

Piece	Classes / content	File it was copied from
Intro card	`p-6 bg-white rounded-xl shadow-md border-l-4 border-indigo-600 flex items-center gap-4`	Products-list header
Circle	`w-12 h-12 rounded-full bg-indigo-100 text-indigo-600 flex items-center justify-center`	Same header; icon swapped to `AddCircleIcon`
Title	`Add a new product`	Replaces the Day 15 `Alert`
Subtitle	`Fill the correct values in the form below`	New copy
Form card	`bg-white p-6 shadow-lg rounded-xl`	So the form is not sitting on the silver/grey canvas

Piece	Classes / content	File it was copied from
Submit	`className="bg-indigo-600 text-white"` on the existing MUI `Button`	Matches the indigo theme

Grid is unchanged from Day 15: two rows, `size={4}` cells, `rowSpacing={4}` `columnSpacing={5}`.

Live demo the class actually ran

Empty **Add Product**:

- Name → `Name is required`
- Description → `Description is required`
- Price → `Price is required`
- Quantity → `Quantity is required`

Then, field by field:

What they typed	What appeared
One character in Name	`Minimum 3 characters are required`
Three characters in Name	Red state **cleared**
Description longer than 100	`Maximum 100 characters allowed`
Negative price (`-7`)	`Price cannot be negative` (then the reworded `-ve or 0` message)
Letters in Price	First: Yup's technical *must be a* `number` *type*. After `.typeError` : `Only numeric value is allowed`
`0` in Price	Not positive → same "cannot be negative / zero" message
`0` in Quantity	`Quantity must be atleast 1`
Working set: name **Kurkure**, description **snacks** / *₹10 के तुरतुरे*, price **20**, qty **23**	Toast **Product Created** — `onSubmit` ran, `POST /product` succeeded, `reset()` cleared the boxes

The lecturer's library analogy, once the Controller boilerplate felt heavy: *it's like a sports bike. पहली बार कोई महंगी bike थमा दी है तो सोचोगे कि अब ये इस बटन से क्या होगा. शुरू में time लगेगा. एक बार bike चलाना सीख गए, तो फिर हमसे बड़ा छपरी दुनिया में कोई नहीं है.* Same claim for a library: the first wiring is the worst wiring.

> **Exam:** `Controller` is the wrapper that makes an unknown UI widget look like a React Hook Form field. Props: `name`, `control`, `render`. Spread `{...field}` onto `TextField`. `error` is boolean (`!!errors.name`). `helperText` is `errors.name?.message`. Do not put `name` / `value` / `onChange` on the MUI field yourself.

> **Pitfall:** Forgetting `{...field}` gives you a pretty box that React Hook Form is not watching. Submit will treat that field as empty (or missing) even if the user typed. Forgetting `control={control}` is the same class of bug: the Controller is not registered on this form.

Login Screen

Problem: the backend already has auth — the UI has nowhere to type it

The lecturer: *we have created the authentication flow at the backend side. So now it's time to integrate that same process at the front end as well.* This sitting is only the **screen**. The remaining fifteen minutes were not enough for the API or the token. His own split:

This morning	Evening (promised, not built)
`Login` page, Tailwind layout, Phone + Password boxes, Login button	Call the login API
Route `/login` **outside** `AppLayout`	Token-related work on the frontend

Do not invent a `POST /login` or a `localStorage` token in these notes. That code is not in `WPT Day 16 Morning Codes`.

Mechanism: a page that is not inside the chrome

New folder and file:

`WPT Day 16 Morning Codes/crud-ui/src/pages/Login/Login.tsx`

Function component, **not** wrapped in `memo`. No `useCallback`, no `useMemo`. The lecturer: *I'm leaving it to you guys only.* Same homework tone as last night's `UpdateProduct`.

The layout is two `Box`es, the CSS-centering pattern they already know, now as Tailwind utilities.

```
browser viewport
┌──────────────── outer Box  min-h-screen bg-gray-100 ────────────────┐
│                  flex items-center justify-center                   │
│                                                                     │
│        ┌──── inner Box  w-full max-w-md bg-white ────┐              │
```

```
|        |  lock icon in an indigo circle                  |          |
|        |  "Login to continue with CRUD UI"               |          |
|        |  Phone TextField                                |          |
|        |  Password TextField                             |          |
|        |  Login Button                                   |          |
|        └─────────────────────────────────────────────────┘          |
└──────────────────────────────────────────────────────────────────────┘
```

Class on the outer box	Job
`min-h-screen`	Occupy the **full** viewport height, not just the card's height
`bg-gray-100`	Grey canvas (the "silver" behind the card)
`flex`	Required before the next two classes do anything
`items-center`	Vertical centre
`justify-center`	Horizontal centre

Inner card, after they tried a fixed `w-100` and then switched:

Class	Job
`w-full`	Grow up to the parent
`max-w-md`	Cap at Tailwind's **medium** max-width: **448 px** / **28 rem**. Lecturer looked up `max-width` in the Tailwind docs for this.
`bg-white`	White card on the grey canvas
`p-8`	Padding 8 × 4 px = **32 px**
`rounded-xl`	Large radius
`shadow-lg`	Large shadow

The lock circle is the products-list / create-product disc again (`w-14 h-14` this time, slightly larger), icon swapped to `LockIcon` from `@mui/icons-material/Lock` . He looked at Material Icons for a login-specific glyph and settled on the lock.

Heading, immediately under the icon:

```
<Typography className="mt-4 text-2xl font-bold text-center text-gray-800">
    Login to continue with CRUD UI
</Typography>
```

Form under that is another `Box component="form"` with `noValidate` (same reason as create — they will put React Hook Form on this later; the browser must not steal validation). Layout this time is **not** the MUI Grid. Lecturer: *By default, if you give display flex, all the input fields now is going to be aligned into the same row. What I want is each input field into a new row.* So:

```
<Box component="form" noValidate className="mt-8 flex flex-col gap-6">
```

`flex-col` stacks Phone, Password, Login. `gap-6` is the space between them.

The two `TextField`s are **plain** MUI — no `Controller` yet. The file even leaves the create-page error lines commented (`// {...field}` , `// error={!!errors.description}`) as a reminder of what evening will paste in. Labels: **Phone** (they log in with phone number on the backend) and **Password**.

Submit button:

```
<Button
    variant="contained"
    type="submit"
    fullWidth
    className="bg-indigo-600 rounded-lg text-white"
>
    Login
</Button>
```

`fullWidth` so it matches the fields. Indigo 600 is the same theme token as Add Product / the AppBar.

`/login` must not render inside `AppLayout`

`WPT Day 15 Evening Codes/crud-ui/src/components/AppRouter/AppRouter.tsx` has every page as a **child** of `<Route path="/" element={<AppLayout />}>`. Children of that route get the sidenav and the top bar through `AppLayout`'s `Outlet`.

A login screen with a sidenav is the wrong product. The lecturer: *ये App Layout अब वो login पे काम नहीं करेगा. It should be an independent screen. वो App Layout के अंदर नहीं render होगा.*

Morning `AppRouter` — `WPT Day 16 Morning Codes/crud-ui/src/components/AppRouter/AppRouter.tsx` — adds a **sibling** route, not a nested one:

```
<Routes>
    <Route path="/login" element={<Login />} />
    <Route path="/" element={<AppLayout />}>
        <Route path="/" element={<Dashboard />} />
        <Route path="/products-list" element={<ProductsList />} />
```

```
        <Route path="/add-product" element={<CreateProduct />} />
        <Route path="/edit-product/:id" element={<UpdateProduct />} />
    </Route>
</Routes>
```

Visit `http://localhost:5173/login` (Vite still on **5173**). You should see the grey full-page canvas and the white card in the middle — no drawer, no AppBar.

The icon on the card is **not** centred in the finished file. The lecturer said they would bring it to the centre in the evening session. Do not "fix" that in your head as if it was already done.

> **Exam:** Login is a standalone route: `<Route path="/login" element={<Login />} />` **next to** the `AppLayout` route, not inside it. Outer box: `min-h-screen bg-gray-100 flex items-center justify-center`. Inner card: `w-full max-w-md` (448 px). Phone + Password + Login. No API and no token this morning.

> **Pitfall:** Nesting `/login` under `AppLayout` puts the sidenav on the login page. Using only `bg-gray-100` without `min-h-screen` paints grey behind the card's own height and leaves the rest of the viewport white. Using `flex` without `flex-col` puts Phone and Password on one row.

Combined Worked Example

Four files that, together, are this morning. They sit under `WPT Day 16 Morning Codes/crud-ui/`. `ApiClient.ts` and `ToastUtil.ts` are unchanged from Day 15 — still `POST /product` to port **7500**, still `showSuccessToast("Product Created !")`.

`src/pages/CreateProduct/types.ts`

```ts
export interface IFormData {
    name: string;
    description: string;
    price: string;
    qty: string;
}
```

`src/pages/CreateProduct/ValidationSchema.ts`

```ts
import { number, object, string } from "yup";
```

```
export const productValidationSchema = object({
    name: string()
        .required("Name is required")
        .min(3, "Minimum 3 characters are required"),

    description: string()
        .required("Description is required")
        .max(100, "Maximum 100 characters allowed"),

    price: number()
        .typeError("Only numeric value is allowed")
        .required("Price is required")
        .positive("Price cannot be -ve or 0"),

    qty: number()
        .required("Quantity is required")
        .integer("Quantity must be integer")
        .min(1, "Quantity must be atleast 1")
});
```

src/pages/CreateProduct/CreateProduct.tsx

```
import { Box, Button, Grid, TextField, Typography } from "@mui/material";
import { memo } from "react";
import type { IFormData } from "./types";
import { apiClient } from "../../api-client/ApiClient";
import { showErrorToast, showSuccessToast } from "../../utils/ToastUtil";
import AddCircleIcon from "@mui/icons-material/AddCircle";
import { Controller, useForm } from "react-hook-form";
import { yupResolver } from "@hookform/resolvers/yup";
import { productValidationSchema } from "./ValidationSchema";

export const CreateProduct = memo(() => {
    const {
        control,
        reset,
        formState: { errors },
        handleSubmit
    } = useForm<IFormData>({
        resolver: yupResolver(productValidationSchema)
    });

    const onSubmit = async (formData: IFormData) => {
        console.log(formData);
        try {
            const response = await apiClient.post("/product", formData);
            console.log(response);
            console.log(response.data);
            showSuccessToast("Product Created !");
```

```
            reset();
        } catch (error) {
            console.log(error);
            showErrorToast("Product Creation Failed !");
        }
    };

    return (
        <Box>
            <Box className="p-6 bg-white rounded-xl shadow-md border-l-4 border-
indigo-600 flex items-center gap-4">
                <Box className="w-12 h-12 rounded-full bg-indigo-100 text-indigo-600
flex items-center justify-center">
                    <AddCircleIcon />
                </Box>
                <Box>
                    <Typography className="text-2xl font-bold text-gray-800">
                        Add a new product
                    </Typography>
                    <Typography className="text-sm text-gray-500">
                        Fill the correct values in the form below
                    </Typography>
                </Box>
            </Box>
            <Box
                component="form"
                className="bg-white p-6 shadow-lg rounded-xl"
                onSubmit={handleSubmit(onSubmit)}
                noValidate
                sx={{ marginTop: "30px" }}
            >
                <Grid container rowSpacing={4} columnSpacing={5} sx={{ mb: 4 }}>
                    <Grid size={4}>
                        <Controller
                            name="name"
                            control={control}
                            render={(controller) => {
                                const { field } = controller;
                                return (
                                    <TextField
                                        {...field}
                                        label="Name"
                                        variant="outlined"
                                        fullWidth
                                        error={!!errors.name}
                                        helperText={errors.name?.message}
                                    />
                                );
                            }}
                        />
                    </Grid>
```

```
                    <Grid size={4}>
                        <Controller
                            name="description"
                            control={control}
                            render={(controller) => {
                                const { field } = controller;
                                return (
                                    <TextField
                                        {...field}
                                        label="Description"
                                        variant="outlined"
                                        fullWidth
                                        error={!!errors.description}
                                        helperText={errors.description?.message}
                                    />
                                );
                            }}
                        />
                    </Grid>
                </Grid>
                <Grid container rowSpacing={4} columnSpacing={5} sx={{ mb: 4 }}>
                    <Grid size={4}>
                        <Controller
                            name="price"
                            control={control}
                            render={({ field }) => (
                                <TextField
                                    {...field}
                                    label="Price"
                                    variant="outlined"
                                    fullWidth
                                    error={!!errors.price}
                                    helperText={errors.price?.message}
                                />
                            )}
                        />
                    </Grid>
                    <Grid size={4}>
                        <Controller
                            name="qty"
                            control={control}
                            render={({ field }) => (
                                <TextField
                                    {...field}
                                    label="Quantity"
                                    variant="outlined"
                                    fullWidth
                                    error={!!errors.qty}
                                    helperText={errors.qty?.message}
                                />
                            )}
```

```
                        />
                    </Grid>
                </Grid>
                <Button type="submit" variant="contained" className="bg-indigo-600
text-white">
                    Add Product
                </Button>
            </Box>
        </Box>
    );
});
```

src/pages/Login/Login.tsx

```
import { Box, Button, TextField, Typography } from "@mui/material";
import LockIcon from "@mui/icons-material/Lock";

export function Login() {
    return (
        <Box className="min-h-screen bg-gray-100 flex items-center justify-center">
            <Box className="w-full max-w-md bg-white p-8 rounded-xl shadow-lg">
                <Box className="w-14 h-14 rounded-full bg-indigo-100 text-indigo-600
flex items-center justify-center">
                    <LockIcon />
                </Box>
                <Typography className="mt-4 text-2xl font-bold text-center text-gray-
800">
                    Login to continue with CRUD UI
                </Typography>
                <Box component="form" noValidate className="mt-8 flex flex-col gap-6">
                    <TextField label="Phone" variant="outlined" fullWidth />
                    <TextField label="Password" variant="outlined" fullWidth />
                    <Button
                        variant="contained"
                        type="submit"
                        fullWidth
                        className="bg-indigo-600 rounded-lg text-white"
                    >
                        Login
                    </Button>
                </Box>
            </Box>
        </Box>
    );
}
```

`src/components/AppRouter/AppRouter.tsx`

```tsx
import { Route, Routes } from "react-router-dom";
import { AppLayout } from "../AppLayout/AppLayout";
import { Dashboard } from "../../pages/Dashboard/Dashboard";
import { ProductsList } from "../../pages/ProductsList/ProductsList";
import { CreateProduct } from "../../pages/CreateProduct/CreateProduct";
import { memo } from "react";
import { UpdateProduct } from "../../pages/UpdateProduct/UpdateProduct";
import { Login } from "../../pages/Login/Login";

export const AppRouter = memo(() => {
    return (
        <Routes>
            <Route path="/login" element={<Login />} />
            <Route path="/" element={<AppLayout />}>
                <Route path="/" element={<Dashboard />} />
                <Route path="/products-list" element={<ProductsList />} />
                <Route path="/add-product" element={<CreateProduct />} />
                <Route path="/edit-product/:id" element={<UpdateProduct />} />
            </Route>
        </Routes>
    );
});
```

End-to-end trace — Kurkure is created only after the schema says yes

1. `npm i react-hook-form yup @hookform/resolvers` has already been run in `crud-ui`. Vite is on **5173**, API on **7500**.
2. Sidenav → Add Product. `AppRouter` matches `/add-product` **inside** `AppLayout`. Sidenav and top bar are visible. The indigo header card reads **Add a new product**.
3. User clicks **Add Product** with every box empty. Browser default validation does not run (`noValidate`). `handleSubmit` runs `productValidationSchema`. All four keys land in `errors`. Each `Controller` sets `error={true}` and shows that field's `.required` message. `onSubmit` is **not** called. Network tab has no `POST`.
4. Name = `K`. `required` passes, `min(3)` fails. Helper text becomes `Minimum 3 characters are required`.
5. Name = `Kurkure` (or the class's spelling). Name key drops out of `errors`. Red clears on that box only.
6. Description typed past 100 characters. `errors.description.message` is `Maximum 100 characters allowed`.
7. Price = `abc`. Without `typeError` the helper text is Yup's technical number-type sentence. With the line class added: `Only numeric value is allowed`.

8. Price = `-7`. `positive` fails: `Price cannot be -ve or 0`.
9. Price = `0`. Same message — zero is not positive.
10. Price = `20`. Price key drops out of `errors`.
11. Quantity = `0`. `min(1)` fails: `Quantity must be atleast 1`. Quantity = `2.5` would fail `integer` instead.
12. Quantity = `23`. Schema now passes. `handleSubmit` calls `onSubmit({ name, description, price, qty })`.
13. `apiClient.post("/product", formData)` hits Express. Success toast **Product Created !**. `reset()` blanks the four Controllers. No `setFormData`.

End-to-end trace — `/login` is a different kind of route

1. Address bar: `/login`.
2. `AppRouter` matches the **first** `<Route>`, the one whose `element` is `<Login />`. `AppLayout` does not mount. No sidenav, no AppBar.
3. Outer `Box` paints `bg-gray-100` for at least the viewport (`min-h-screen`) and flex-centres the inner card.
4. Inner card is at most `max-w-md` (448 px), white, padded, rounded, shadowed. Lock disc, heading **Login to continue with CRUD UI**, Phone, Password, indigo **Login**.
5. Clicking **Login** does not call an API this morning. The form has `noValidate` and no `onSubmit={handleSubmit(…)}` yet. Evening session is where that wiring + the token work was promised.

What Day 15 Evening `CreateProduct.tsx` still looks like (the before)

Same four fields, **your** state, **your** change handler, **no** schema, **no** red helper text:

```
const [formData, setFormData] = useState<IFormData>({
    name: "", description: "", price: "", qty: ""
});

const handleChange = (
    event: ChangeEvent<HTMLInputElement | HTMLTextAreaElement>
) => {
    setFormData({ ...formData, [event.target.name]: event.target.value });
};

const handleSubmit = async (event: SubmitEvent<HTMLFormElement>) => {
    event.preventDefault();
    const response = await apiClient.post("/product", formData);
    showSuccessToast("Product Created !");
```

```
    setFormData({ name: "", description: "", price: "", qty: "" });
};
```

That file is `WPT Day 15 Evening Codes/crud-ui/src/pages/CreateProduct/CreateProduct.tsx`. Morning's job is to delete the first two blocks, rename the third to `onSubmit(formData)`, and let `useForm` + Yup + `Controller` own the rest.

Quick Revision Checklist

Topic	One-line takeaway
Why validate	Name min 3, description max 100, price numeric and > 0, qty integer ≥ 1 — otherwise the POST stores junk
Quantity picture	You do not sell 3.5 boards; `integer` + `min(1)`
Manual cost	Own `useState`, own `onChange`, own `if` ladder per field
`react-hook-form`	Owns data, change, submit, error display
`useForm`	The one hook; typed `useForm<IFormData>`
Do not write	Field `useState`, `handleChange`, `event.preventDefault`, empty `setFormData`
`handleSubmit`	Library function on the form: `onSubmit={handleSubmit(onSubmit)}`
`onSubmit`	Your function; argument is `formData`, not `event`
`reset()`	Clears registered fields after a good POST
`noValidate`	Browser HTML5 validation stays out of the way
Yup	Rules, not `if`s; package name `yup` (lecture audio "Yep")
Zod	Named alternative schema library; class did not use it
Schema keys	Must equal the input / Controller `name` (`qty`, not `quantity`)
Messages	Second argument of `.required()`, `.min()`, `.positive()`, …
`typeError`	Human sentence when Price is not a number; added after the technical default
`positive`	Greater than zero — **0 fails**; message was reworded to say so

Topic	One-line takeaway
File	`CreateProduct/ValidationSchema.ts` — keep it out of the TSX
`@hookform/resolvers`	Middleman; `yupResolver(productValidationSchema)`
`errors`	Object of **failing** fields; `{ name: { message } }`
`control`	*Connect each field to the form*
`Controller`	Wrapper because RHF does not know MUI `TextField`
`render`	You return the real widget; receive `{ field, fieldState, formState }`
`{...field}`	Spreads `name` / `value` / `onChange` / `onBlur` / `ref`
`error` / `helperText`	MUI boolean + MUI sentence; `!!errors.name` and `errors.name?.message`
`!!`	Force a real boolean; one `!` would flip the answer
Create header	Copied list-page card; `AddCircleIcon`; form card `bg-white shadow-lg rounded-xl`
Demo success	Kurkure / snacks / 20 / 23 → **Product Created**
Login file	`src/pages/Login/Login.tsx` — no `memo` (homework)
Login layout	Grey full-screen flex centre; white `max-w-md` (448 px) card
Login fields	Phone + Password + indigo Login; **no** Controller / API yet
`/login` route	Sibling of `AppLayout`, not a child — no sidenav
Evening leftover	Login API, token, centre the lock icon
Starting code	`WPT Day 15 Evening Codes/…/CreateProduct.tsx` is the manual form
Finished code	`WPT Day 16 Morning Codes/…` has schema, Controllers, Login, new route

Practice Exercises

Concept

1. The lecturer named four kinds of rule before opening any library. Map each rule to the product field it belongs to.

2. Why is `qty: 2.5` illegal in this catalogue? Use the lecture's own picture, not a generic "because it should be an integer."
3. List the five plumbing jobs the Day 15 `CreateProduct.tsx` does by hand that React Hook Form is supposed to take over.
4. What is the **one** hook React Hook Form gives you for the whole form, and which four pieces did class pull out of it?
5. Why did class rename their own function from `handleSubmit` to `onSubmit`?
6. What is the argument of `onSubmit` after the migration, and what two Day 15 lines disappear because of that?
7. What does `noValidate` tell the browser, and why is it needed once Yup is in charge?
8. Schema-based validation vs if-else: what do you write in a schema, and what do you stop writing?
9. Name the three npm packages and give each one its job in one phrase.
10. Why is a third package (`@hookform/resolvers`) required when React Hook Form and Yup already exist?
11. What must be true of every **key** in `productValidationSchema`, and what breaks if you write `quantity` instead of `qty`?
12. Where do the user-visible sentences (`Name is required`, ...) live — in the component, or in the schema — and which function argument are they?
13. `price` used `.positive("Price cannot be negative")` first. Why did the message later become `Price cannot be −ve or 0`?
14. What did a **string** in the Price box show *before* `.typeError`, and what did class change it to?
15. Description's final rule is `max(100)`, not `min(3)`. What is allowed, and what is not?
16. Why was `ValidationSchema.ts` a **new file** instead of a block at the top of `CreateProduct.tsx`?
17. Draw (or list) the shape of `errors` when Name is empty and Price is `"abc"`, and when every field is valid.
18. React Hook Form does not know MUI. What component exists because of that, and what three props does it take?
19. What is inside `field`, and why does class write `{...field}` rather than `name={field.name}` `onChange={field.onChange}`?
20. MUI `error` wants a boolean. `errors.name` is an object or `undefined`. How did class turn one into the other, and what does a single `!` do wrong?
21. Which prop shows the sentence under the box, and why is there a `?.` before `message`?

22. Name vs Price `render` functions in the saved `CreateProduct.tsx` look different. What JavaScript benefit did the lecturer use on Price and Quantity?
23. After a successful POST, Day 15 called `setFormData({ name: "", … })`. What is the one function call that replaces that, and where does it come from?
24. Why is `/login` **not** nested under the `AppLayout` route?
25. `max-w-md` — what actual pixel / rem cap did the lecturer read from the Tailwind docs?
26. `flex` on the login form without `flex-col`: where do Phone and Password sit?
27. What did this morning **not** do with the login form, and which session was that promised for?
28. `Login` is not wrapped in `memo`. Was that an accident?

Application

29. Write the install command from `notes.txt`.
30. Write `productValidationSchema` with the four keys and the messages from the saved `ValidationSchema.ts`.
31. Write the `useForm<IFormData>({ … })` call including the resolver import line.
32. Write `onSubmit` as it is after the migration (signature + POST + success toast + reset + error toast).
33. Write the `Box component="form"` opening tag (the `onSubmit`, `noValidate`, and theme classes).
34. Write one complete `Controller` for `qty` using the implicit-return `render` style, including `error` and `helperText`.
35. Write the `/login` route next to the `AppLayout` route (only those two `Route`s, with the four children left as a comment).
36. Write the outer + inner `Box` classNames for the login layout (no fields, just the two shells).

Predict the output

37. User clicks Add Product with every field empty. Is `apiClient.post` called? What four helper texts appear?
38. Name is `Ku` (two characters), everything else valid. What is `errors.name?.message`? Does `onSubmit` run?
39. Description is 101 characters, everything else valid. What message appears, and on which field?
40. Price is `hello`. Schema still has `.typeError("Only numeric value is allowed")`. What is the helper text?
41. Price is `0`, schema has `.positive("Price cannot be -ve or 0")`. What happens and why?
42. Quantity is `3.6`. Which rule fails, and what is the message in the saved schema?

43. Schema key is `quantity`. Controller `name` is `"qty"`. User leaves quantity blank and submits. What do you see on that box?
44. Form is `<Box component="form" onSubmit={onSubmit} noValidate>` — they passed **their** function, not `handleSubmit(onSubmit)`. Empty submit. What happens?
45. `TextField` has `label="Name"` and `{...field}` was forgotten. User types `Kurkure` and the other three fields are valid. What does `onSubmit` receive for `name` (if it runs at all)?
46. `error={errors.name}` (no `!!`, no ternary). Name is valid. What is React trying to put into a boolean prop?
47. User opens `/login`. Which chrome (sidenav / top bar) is visible, and why?
48. Login outer box is `className="bg-gray-100 flex items-center justify-center"` — no `min-h-screen`. What does the page look like if the card is short?

Spot the bug

49. `import { yupResolver } from "@hookform/resolvers";` — the rest of `useForm` looks like class. Why does it fail?
50. `name: string().required().min(3)` — no message strings. The form refuses submit on a blank name. What is missing on screen?
51. `useForm()` with no generic and no `resolver`. Controllers are in place. User submits `name: "ab"`. Does the POST fire?
52. `const onSubmit = async (event: SubmitEvent<HTMLFormElement>) => { event.preventDefault(); … }` is passed as `handleSubmit(onSubmit)`. What is actually in that first argument at runtime?
53. Login route: `<Route path="/" element={<AppLayout />}><Route path="/login" element={<Login />} /></Route>`. What extra UI appears on `/login`?
54. Quantity Controller: `name="quantity"` while the schema and `IFormData` use `qty`. Name the two lookups that now miss each other.
55. `helperText={errors.name.message}` — no `?.`. Name is valid. What happens on render?

Practice Answers

1. Name: required + min 3 characters. Description: required + (final file) max 100. Price: required + numeric + greater than 0. Quantity: required + numeric + greater than 0 + integer.

2. Quantity is a count of whole items. The lecturer: you are not going to fetch 3.5 boards by snapping one in half. `2.5` is a fraction, so `.integer()` must fail.

3. (1) `useState` for the four values. (2) `handleChange` spreading into that state. (3) `event.preventDefault()`. (4) `setFormData({ …empty })` after success. (5) The `if` ladder that would decide each error message.

4. `useForm`. Class took `control`, `reset`, `formState: { errors }`, and `handleSubmit`.

5. `handleSubmit` is now the **library** function. Keeping the old name collides and hides the wrapper you have to call.

6. `formData: IFormData`. Gone: the `event` parameter and `event.preventDefault()`.

7. Do not run the browser's own HTML5 validation. Otherwise native bubbles fight Yup / React Hook Form.

8. You write **rules** (`string().required().min(3)`). You stop writing the `if (value === "")` / `if (value.length < 3)` / `if (!Number.isInteger(n))` chain.

9. `react-hook-form` — run the form. `yup` — write the schema. `@hookform/resolvers` — link the two.

10. They are independent libraries. Neither one imports the other. The resolver is the middleman the lecturer installed for that reason.

11. Each key must equal the field's `name` / Controller `name`. `quantity` vs `qty` means Yup never validates the box the user is typing in.

12. In the schema, as the argument to `.required("…")`, `.min(3, "…")`, `.max(100, "…")`, `.positive("…")`, `.integer("…")`, `.typeError("…")`.

13. They typed `0`. `positive` rejected it (zero is not > 0) but the old sentence only mentioned negative. Users would not understand why zero failed.

14. Before: Yup's technical *must be a* `number` *type*. After: `Only numeric value is allowed`.

15. A one-character description is allowed. The 101st character is not.

16. So the TSX does not mix UI with the rule list. Lecturer: *it will again mix up few of the things.*

17. Failing: `{ name: { message: "Name is required" }, price: { message: "Only numeric value is allowed" } }` (description/qty omitted if they pass). All valid: `{}` / no keys — `errors.name` is `undefined`.

18. `Controller`. Props: `name`, `control`, `render`.

19. The props the widget needs (`name`, `value`, `onChange`, `onBlur`, `ref`). Spreading applies all of them so you cannot forget one.

20. `!!errors.name` (or `errors.name ? true : false`). A single `!` turns `undefined` into `true`, which would paint a red border on a **valid** field.

21. `helperText` . `?.` so a missing `errors.name` does not throw when you read `.message` .

22. Implicit return: drop `{ }` and `return` , wrap the `TextField` in `( )` . One-expression arrow function.

23. `reset()` , returned by `useForm` .

24. `AppLayout` is sidenav + top bar + `Outlet` . Login is an independent full-page screen. Nesting would draw the chrome around the card.

25. 448 px, also given as **28 rem**.

26. On the **same row**. Default flex direction is row.

27. No login API call, no token handling. Promised for the **evening** session.

28. No. He skipped `memo` / `useCallback` / `useMemo` on purpose and left them as student work.

29.

```
npm i react-hook-form yup @hookform/resolvers
```

30.

```
import { number, object, string } from "yup";

export const productValidationSchema = object({
    name: string()
        .required("Name is required")
        .min(3, "Minimum 3 characters are required"),
    description: string()
        .required("Description is required")
        .max(100, "Maximum 100 characters allowed"),
    price: number()
        .typeError("Only numeric value is allowed")
        .required("Price is required")
        .positive("Price cannot be -ve or 0"),
    qty: number()
        .required("Quantity is required")
        .integer("Quantity must be integer")
        .min(1, "Quantity must be atleast 1")
});
```

31.

```
import { yupResolver } from "@hookform/resolvers/yup";
import { productValidationSchema } from "./ValidationSchema";

const {
```

```
    control,
    reset,
    formState: { errors },
    handleSubmit
} = useForm<IFormData>({
    resolver: yupResolver(productValidationSchema)
});
```

32.

```
const onSubmit = async (formData: IFormData) => {
    try {
        await apiClient.post("/product", formData);
        showSuccessToast("Product Created !");
        reset();
    } catch (error) {
        console.log(error);
        showErrorToast("Product Creation Failed !");
    }
};
```

33.

```
<Box
    component="form"
    className="bg-white p-6 shadow-lg rounded-xl"
    onSubmit={handleSubmit(onSubmit)}
    noValidate
    sx={{ marginTop: "30px" }}
>
```

34.

```
<Controller
    name="qty"
    control={control}
    render={({ field }) => (
        <TextField
            {...field}
            label="Quantity"
            variant="outlined"
            fullWidth
            error={!!errors.qty}
            helperText={errors.qty?.message}
        />
    )}
/>
```

35.

```
<Routes>
    <Route path="/login" element={<Login />} />
    <Route path="/" element={<AppLayout />}>
        {/* dashboard, products-list, add-product, edit-product/:id */}
    </Route>
</Routes>
```

36.

```
<Box className="min-h-screen bg-gray-100 flex items-center justify-center">
    <Box className="w-full max-w-md bg-white p-8 rounded-xl shadow-lg">
    </Box>
</Box>
```

37. No POST. `Name is required`, `Description is required`, `Price is required`, `Quantity is required`.

38. `Minimum 3 characters are required`. `onSubmit` does **not** run.

39. `Maximum 100 characters allowed` on **Description**.

40. `Only numeric value is allowed`.

41. The box goes red with `Price cannot be -ve or 0`. `positive` means greater than zero; `0` fails.

42. `.integer()` — `Quantity must be integer`.

43. The Quantity box stays quiet (no red, no helper text). Yup is watching a key that no Controller registered. `onSubmit` may still be blocked if other fields fail; this field is not part of that decision under the name `qty`.

44. Their `onSubmit` runs immediately. No schema. Empty strings POST to `/product` the Day 15 way (or crash if the function still expects an `event`).

45. React Hook Form never saw the keystrokes. `name` is empty / undefined. Schema `required` fails, or `onSubmit` gets a blank name. Either way **Kurkure is not in form data**.

46. `undefined`. MUI's `error` prop is typed as boolean. That is the bug `!!` was written to fix.

47. Neither. `/login` is a sibling route; `AppLayout` does not mount.

48. Grey only behind the card's own box. The rest of the viewport stays the default page colour. `min-h-screen` is what stretches the grey.

49. `yupResolver` is exported from `@hookform/resolvers/yup`, not from the package root. The import path is wrong.

50. The helper texts. `.required()` still fails the submit, but with no string Yup has nothing friendly to put in `errors.name.message` (default / empty). Class put the sentence in every rule.

51. Yes. Nothing is reading `productValidationSchema`. `"ab"` is a legal POST body as far as the form is concerned.

52. The **form values object**, not a `SubmitEvent`. `event.preventDefault` is then `undefined is not a function` (or a no-op on a plain object). That is why class dropped `event`.

53. Sidenav and top bar. `/login` is now a child of `AppLayout`.

54. Yup looks up `qty`. The Controller publishes `quantity`. `errors.qty` is filled when the schema fails, but the TextField reads `errors.quantity` — both sides miss.

55. Render throws: cannot read `message` of `undefined`. The optional chain is not decorative.